ROBOTICS SUMMER SCHOOL - FUNDAMENTALS

# Visual SLAM

Tracking, mapping, loop closure, and learned geometry

Krish Pandya

Morning session

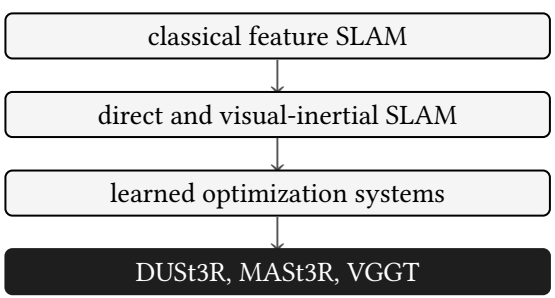
# Storyline: from classical to learned

We start from the same SfM problem:

$$u_{ij} \approx \pi\left(K_i T_i X_j\right)$$

Visual SLAM asks for the online version:

- track now,
- update a map,
- recover after failure,
- close loops,
- keep compute bounded.



# A rough system timeline

Classical and geometric systems:

- PTAM: keyframes and local mapping.
- LSD-SLAM and DSO: direct photometric tracking.
- ORB-SLAM: feature tracking, local BA, loop closure.
- ORB-SLAM3: visual, visual-inertial, multi-map.

Learned and foundation-model systems:

- DUSt3R: two-view point maps.
- MASt3R: point maps plus 3D-aware matching.
- MASt3R-SLAM: learned geometry in a SLAM loop.
- DROID-SLAM: learned dense update with BA-like refinement.
- VGGT: feed-forward multi-view geometry.
- VGGT-SLAM: VGGT local submaps plus graph optimization.
- pi3 and Scal3R: scaling multi-view prediction.

# Visual odometry and visual SLAM

**VISUAL ODOMETRY**

Estimate camera motion from recent images:

$$I_1, \ldots, I_t \rightarrow T_{wc_t}$$

It can run without a persistent map, but drift grows over time.

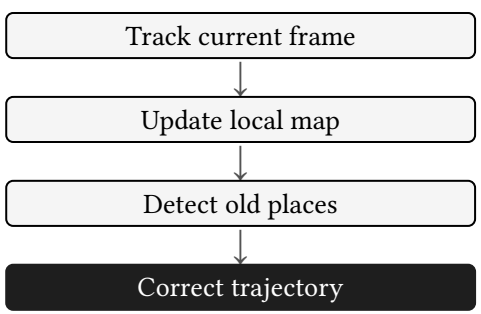
**VISUAL SLAM**

Estimate camera trajectory and a reusable map:

$$I_1, \ldots, I_t \rightarrow \left(\left\{T_{wc_i}\right\}, \left\{X_j\right\}\right)$$

VO is enough for short motion. SLAM is needed when the robot must reuse old observations, recover from tracking loss, or remove drift.

SLAM adds memory. The system can revisit an old place, relocalize, and correct accumulated drift.




Output of a complete system:

- camera trajectory,
- sparse or dense map,
- place database,
- uncertainty or confidence.

# The visual SLAM state

A sparse visual SLAM state:

$$x = \left( \{T_i\}_{\{i \in \mathcal{K}\}}, \{X_j\}_{\{j \in \mathcal{P}\}} \right)$$

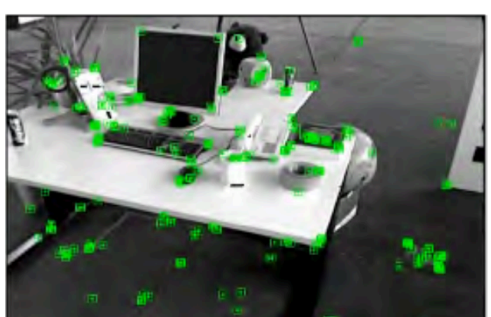
For visual-inertial SLAM:

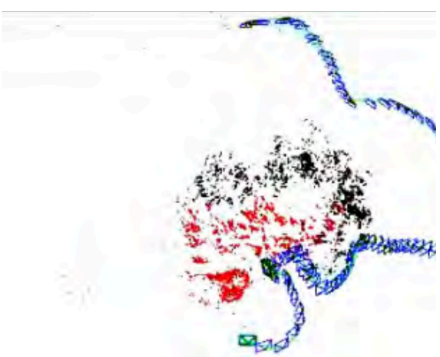
$$x_i = (R_i, p_i, v_i, b_i^g, b_i^a)$$

The map stores:

- keyframes,
- landmarks or inverse-depth points,
- descriptors or tracks,
- graph edges.

A map point is useful only if the system also knows which keyframes observe it and how reliable those observations are.





# The recursive problem

At time $t$, the system has a prior from the map:

$$p(x_t \mid z_{1:t-1}, u_{1:t})$$

The image gives new measurements:

$$z_t = \{u_{tj}\}$$

The backend forms:

$$p(x_t \mid z_{1:t}) \propto p(z_t \mid x_t) p(x_t \mid z_{1:t-1})$$

> In practice this posterior is optimized as a local least-squares problem, not stored as a full probability distribution.

Measurement model:

$$u_{tj} = \pi\left(K T_t X_j\right) + \varepsilon_{tj}$$

Motion prior:

$$T_t \approx T_{t-1} \Delta T_t$$

Optimized variables may be only the current pose, a local keyframe window, or the full pose graph after loop closure.

# Front-end and back-end

**FRONT-END**

- detect or predict measurements,
- associate measurements with map points,
- reject outliers,
- decide keyframes,
- propose loop closures.

Front-end errors are discrete: a match is right or wrong, a keyframe is kept or dropped, a loop candidate is accepted or rejected.

**BACK-END**

- optimize current pose,
- optimize local map,
- maintain priors,
- optimize pose graph,
- fuse IMU and other sensors.

> Most failures are data association failures before they are optimization failures.

Back-end errors are continuous: pose, point, velocity, bias, and scale move by small increments until residuals become small.

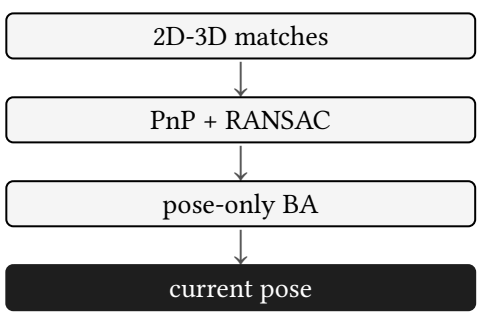
# Tracking with a known local map

When landmarks are known, pose tracking is PnP plus refinement:

$$\min_{T} \sum_{j} \| u_j - \pi\left(K T X_j\right) \|^2_{\Sigma_j^{-1}}$$

RANSAC selects a pose hypothesis. Nonlinear refinement improves it.

A map point is useful only if it projects into the current image and has a consistent descriptor or patch.



# Monocular initialization

Two images give two hypotheses:

- homography $H$ for planar scenes or pure rotation,
- fundamental matrix $F$ for general 3D structure.

After choosing a model:

$$E = K^T F K$$

Decompose $E$ into rotation and translation direction, triangulate points, and run a small BA problem.

Failure cases:

- pure rotation,
- too little baseline,
- moving foreground,
- repeated texture,
- wrong focal length.

Monocular initialization creates structure and scale up to an arbitrary factor.

# If a monocular camera only rotates in place, why can it estimate orientation but not depth?

# Keyframes

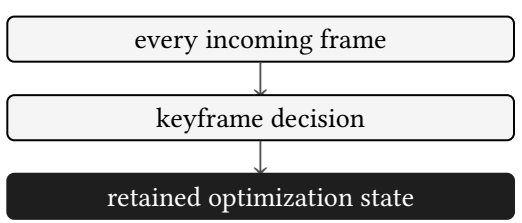
A keyframe is an image retained as a state variable.
New keyframes are added when:

- parallax is large enough,
- tracking quality falls,
- new scene area appears,
- the local map needs coverage.

Keeping every frame makes BA too large.

The keyframe rule trades accuracy against latency. Too many keyframes slow local BA. Too few keyframes weaken tracking.

Keyframes are the compute budget of visual SLAM.



# Covisibility graph

ORB-SLAM uses a covisibility graph:

- nodes are keyframes,
- edge weight is the number of shared map points,
- local tracking queries nearby keyframes,
- local BA optimizes nearby keyframes and points.

Essential graph is a thinner graph used for loop correction.

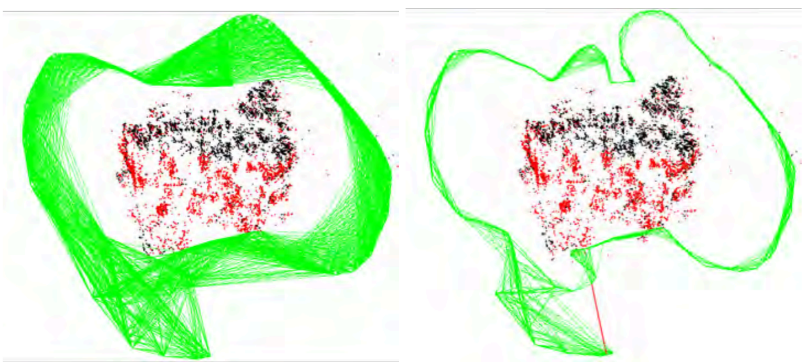


Figure 7.5 Representation of locality in ORB-SLAM [787]. The covisibility graph (left) connects keyframes that have seen at least $\theta$ points in common (in this example $\theta = 15$) and is used for local BA. The essential graph (right) is a sparser version, that in this example connects keyframes with at least $\theta = 100$ points in common, and is used for pose-graph optimization during loop correction. ©2015 IEEE.

# Local BA

Local BA solves:

$$\min_{\{T_i\},\{X_j\}} \sum_{(i,j)\in\mathcal{O}_l} \|u_{ij} - \pi\big(KT_iX_j\big)\|^2$$

It updates:

- current keyframe,
- covisible keyframes,
- points observed by those keyframes.

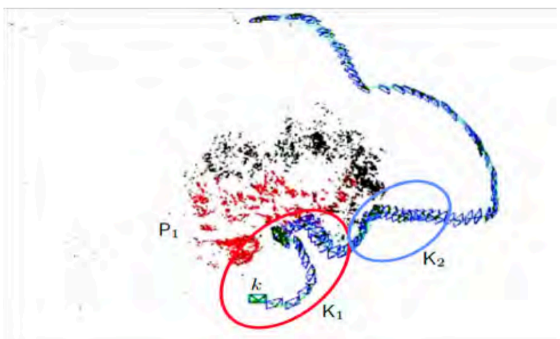



Figure 7.6 Implementation of local BA in ORB-SLAM [787]. The local map is defined by the set of keyframes $\mathsf{K}_1$ that contains the current keyframe $k$ and its neighbors in the covisibility graph, and the set $\mathsf{P}_1$ of points seen by them (in red). $\mathsf{K}_2$ is the rest of keyframes in the map that see some point from $\mathsf{P}_1$.

Local BA is the main difference between a tracker and a map-building system.

# ORB-SLAM system structure

Threads:

- tracking,
- local mapping,
- loop closing,
- optional full BA.

Map data:

- keyframes,
- map points,
- covisibility graph,
- spanning tree,
- place database.

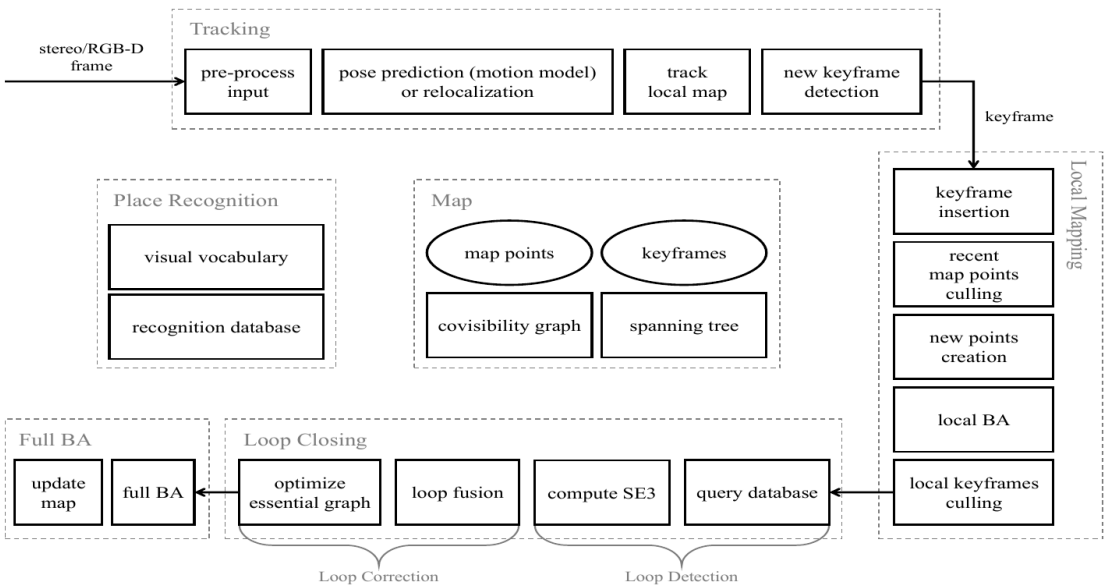



Figure 7.7 Structure of ORB-SLAM2 system [785] showing the map, the place recognition database and its complete processing pipeline composed of four threads: tracking, local mapping, loop closing and full BA. ©2017 IEEE.

# Place recognition

The system asks whether the current frame sees an old place.

Classical pipeline:

- bag of visual words,
- retrieve candidate keyframes,
- match features,
- verify geometry,
- estimate $\mathrm{SE}(3)$, $\mathrm{Sim}(3)$, or another correction.

False positives are expensive. A bad loop edge can bend the entire map.

Verification usually requires geometric consistency:

$$u_b \approx \pi\left(K T_{ba} \Pi^{-1}(u_a, d_a)\right)$$

> Place recognition is not enough. Loop closure needs geometry.

# Loop closure as pose graph optimization

Edges encode relative pose constraints:

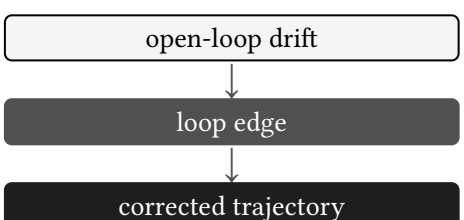
$$Z_{ij} \approx T_i^{-1} T_j$$

Residual:

$$r_{ij} = \mathrm{Log}\left(Z_{ij}^{-1} T_i^{-1} T_j\right)$$

Objective:

$$\min_{\{T_i\}} \sum_{(i,j)\in\mathcal{E}} r_{ij}^T \Omega_{ij} r_{ij}$$



# Relocalization

Relocalization estimates the current pose inside an existing map.

Steps:

- retrieve candidate keyframes,
- match current features to map points,
- solve PnP with RANSAC,
- refine with pose-only BA,
- resume local tracking.

Used when:

- tracking is lost,
- robot starts in a known map,
- camera motion is fast,
- image blur or occlusion breaks recent tracking.

> Relocalization is loop closure at the current frame.

# Direct methods

Direct methods minimize photometric residuals:

$$r_p = I_{i(p)} - I_{j(\pi(T_{ji}\Pi^{-1}(p,d_p)))}$$

Objective:

$$\min_{T,d} \sum_{p \in \mathcal{P}} \rho\left(r_p^2\right)$$

They use image intensity rather than sparse descriptors.

Direct methods need:

- calibrated exposure or brightness model,
- enough image gradient,
- good initial pose,
- static scene assumptions.

Bad lighting or rolling shutter can dominate the residual.

# Visual-inertial SLAM

Add IMU state:

$$x_i = (R_i, p_i, v_i, b_i^g, b_i^a)$$

IMU factors connect adjacent keyframes. Vision factors connect poses to landmarks.

IMU improves scale, gravity, and short-term motion prediction.

220 *Visual SLAM*

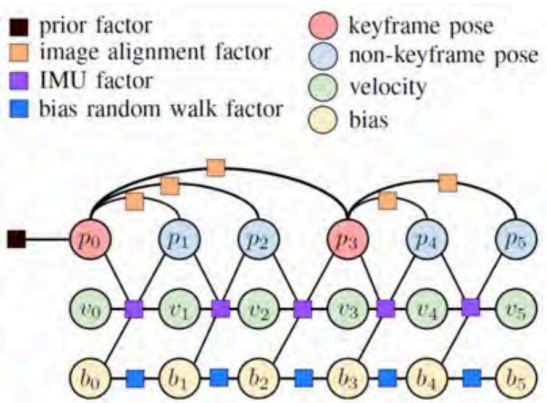



Figure 7.14 The integration of multiple sensors can be elegantly performed in tigl coupled manner using factor graphs. This is an example factor graph for tightly coup fusion of vision and IMU [1116]. It significantly increases precision and robustness camera motion and 3D structure —see Figure 7.15. (©2016 IEEE)

# Observability

Some quantities cannot be inferred from the data.

Monocular without IMU:

- global position is unobservable,
- global yaw depends on chosen frame,
- metric scale is unobservable.

Visual-inertial with enough motion:

- scale becomes observable,
- gravity becomes observable,
- bias can be estimated.

Degenerate motion:

- pure rotation gives no depth,
- constant velocity weakens accelerometer bias estimation,
- planar scenes can make homography dominate,
- repeated texture corrupts data association.

> No optimizer can estimate a variable that the measurements never constrain.

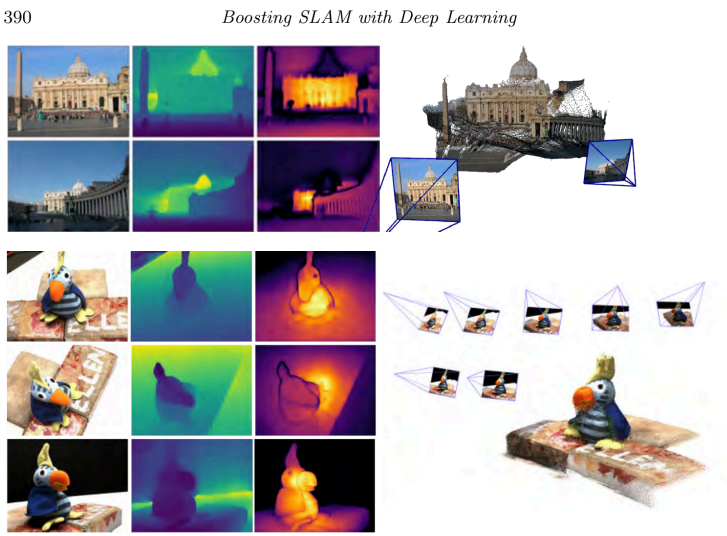
# Learned visual SLAM

Classical V-SLAM builds geometry from tracked features. Learned systems change the measurement source:

- learned features,
- learned matching,
- optical flow,
- point maps,
- depth priors,
- camera priors.

The backend still needs multi-view agreement over time.

The main question becomes: what should the network predict, and what should the optimizer still enforce?



390 *Boosting SLAM with Deep Learning*

Figure 13.15 **Reconstruction examples** on two scenes never seen during training. From left to right: RGB, depth map, confidence map, reconstruction. The top scene shows the raw result output from $f(I^1, I^2)$. The both scene shows the outcome of global alignment (Sec. 13.4.4) Image from [1164] (©2024 IEEE).

# DUSt3R

DUSt3R predicts dense 3D point maps from two images.
Instead of starting from sparse keypoints, it outputs:

- point map for image 1,
- point map for image 2,
- confidence weights.

Both point maps live in the coordinate frame of the first image. This gives a learned two-view geometry primitive.

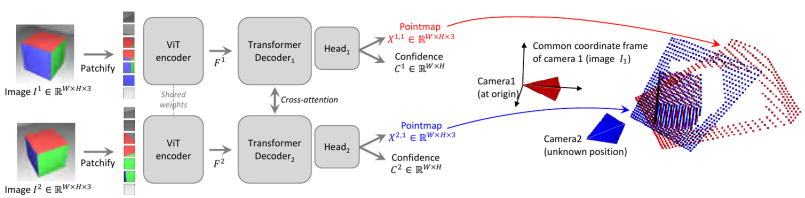



Figure 13.14 **Architecture of the network.** Two views of a scene $(I^1, I^2)$ are first encoded in a Siamese manner with a shared ViT encoder. The resulting token representations $F^1$ and $F^2$ are then passed to two transformer decoders that constantly exchange information via cross-attention. Finally, two regression heads output the two corresponding pointmaps and associated confidence maps. Importantly, the two pointmaps are expressed in the same coordinate frame of the first image $I^1$. The network is trained using a simple regression loss (Eq. (13.17)). Image from [1164] (©2024 IEEE).

# MASt3R and MASt3R-SLAM

MASt3R adds 3D-aware matching to DUSt3R-style point maps.
MASt3R-SLAM turns this pairwise prediction into a running SLAM system:

- select keyframes,
- fuse local geometry,
- track the current frame,
- relocalize after failure,
- close loops,
- correct the map globally.

The network gives strong local geometry. The SLAM system gives memory.



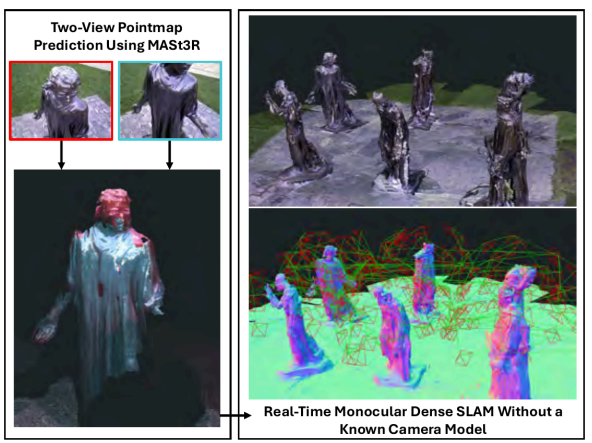



Figure 13.17 Reconstruction from the dense monocular SLAM system on the Burghers sequence [1296]. Using the two-view predictions from MASt3R shown on the left, the system achieves globally reconstructions in real-time without a known camera model [789] (©2025 IEEE).

# DROID-SLAM

DROID-SLAM uses a learned update operator with differentiable BA.
It builds a frame graph and uses dense correspondence to update:

- camera poses,
- inverse depth,
- confidence weights.

This is closer to an optimizer unrolled through a network than a feature-matching front-end.



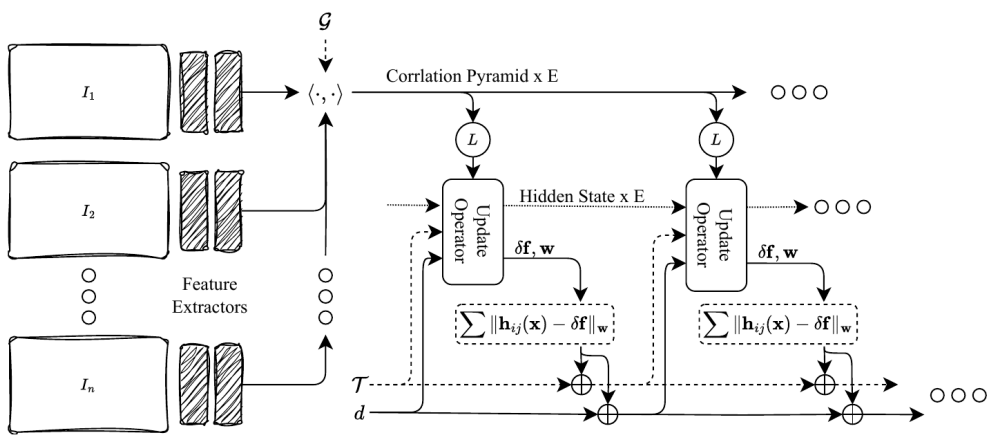



Figure 13.10 DROID-SLAM architecture: Features are extracted from the set of $N$ input images and used to construct $E$ correlation volumes (one for each edge in the frame graph). The update operator acts symmetrically on each edge in the frame graph. In each iteration, it uses the current estimate of the poses $\boldsymbol{T}_1, ..., \boldsymbol{T}_n$ and depths $\boldsymbol{d}_1, ..., \boldsymbol{d}_n$ to index from the correlation pyramid. The correlation features and hidden state are used to produce a flow revision $\delta \boldsymbol{f}$ and confidence weights $\boldsymbol{w}$ which parametrize the cost function. A couple Gauss-Newton updates are applied to generate pose and depth updates.

# VGGT

VGGT predicts several 3D quantities in one feed-forward pass:

- camera intrinsics and extrinsics,
- depth maps,
- point maps,
- point tracks.

It shifts the primitive from pairwise matching to multi-image prediction.

Figure 1. **VGGT** is a large feed-forward transformer with minimal 3D-inductive biases trained on a trove of 3D-annotated data. It accepts up to hundreds of images and predicts cameras, point maps, depth maps, and point tracks for all images at once in less than a second, which often outperforms optimization-based alternatives without further processing.

# VGGT architecture idea

VGGT uses transformer attention over input views and predicts geometry heads.
Important outputs for SLAM:

- camera estimates for local windows,
- dense point maps,
- tracks that can seed correspondence,
- depth maps for dense mapping.

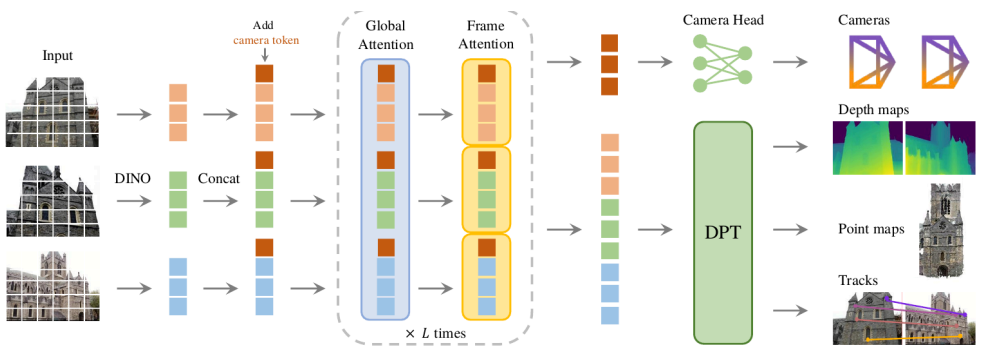



Figure 2. **Architecture Overview.** Our model first patchifies the input images into tokens by DINO, and appends camera tokens for camera prediction. It then alternates between frame-wise and global self attention layers. A camera head makes the final prediction for camera extrinsics and intrinsics, and a DPT [87] head for any dense output.

# VGGT-SLAM

VGGT-SLAM makes local VGGT predictions into a full SLAM system.
It builds submaps from local windows, registers neighboring submaps, adds loop constraints, and optimizes the global submap graph.

Because uncalibrated reconstructions can have projective ambiguity, the system uses $\mathrm{SL}(4)$ transforms between submaps.

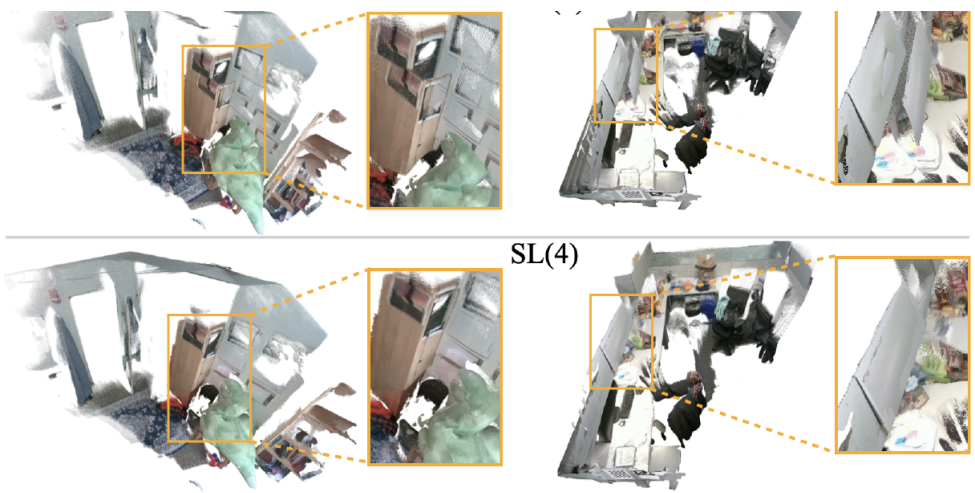



Figure 1: VGGT-SLAM alignment of 6 submaps created from VGGT using $\mathrm{Sim}(3)$ alignment (top) and $\mathrm{SL}(4)$

# Why $\mathrm{SL}(4)$ appears

Similarity transform:

$$X' = sRX + t$$

Projective transform in homogeneous coordinates:

$$\tilde{X}' = H\tilde{X}, \quad H \in \mathrm{SL}(4)$$

$\mathrm{SL}(4)$ has 15 degrees of freedom after fixing determinant.

Feed-forward 3D prediction can reintroduce projective ambiguity.
The modern model brings back a classical geometry problem.

Submap graph objective:

$$\min_{\{H_i\}} \sum_{(i,j)} \left\| \mathrm{Log}\left( H_{ij}^{-1} H_i^{-1} H_j \right) \right\|_{\Omega_{ij}}^2$$

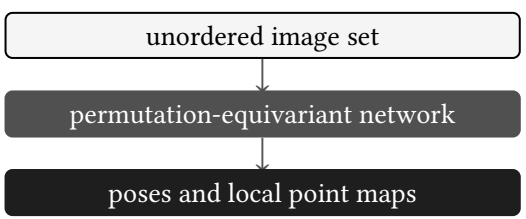
# pi3

pi3 focuses on permutation-equivariant visual geometry. The target property:

$$f(P_\pi I) = P_\pi f(I)$$

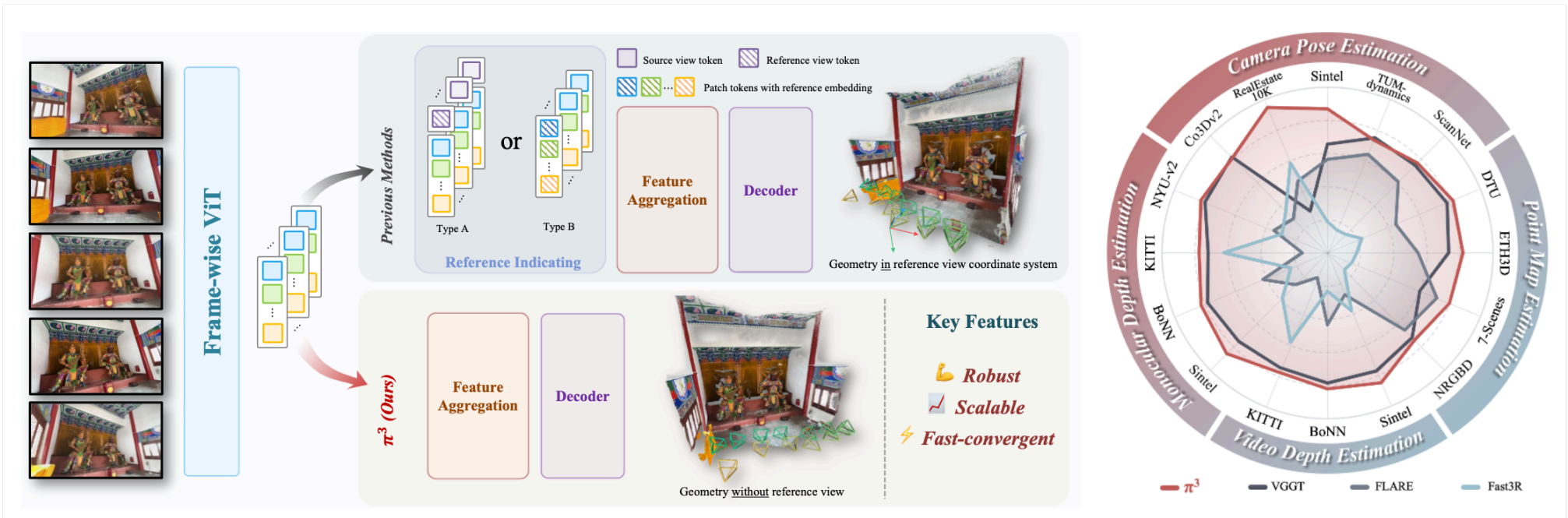
The reconstruction should not depend on the arbitrary order of input views.

This matters for unordered photos and SLAM windows where frame selection can vary.




No fixed reference frame means the output ordering follows the input ordering, instead of depending on one chosen anchor image.

# pi3 architecture visual




Reference-view methods pick or encode an anchor image. pi3 removes that anchor and keeps the prediction equivariant to view order.

# Scal3R

Scal3R targets long RGB sequences.
It partitions a sequence into overlapping chunks, processes chunks, shares global context through memory, and fuses the chunk reconstructions.

This is the scaling problem for VGGT-like methods: attention cost grows quickly with the number of frames.

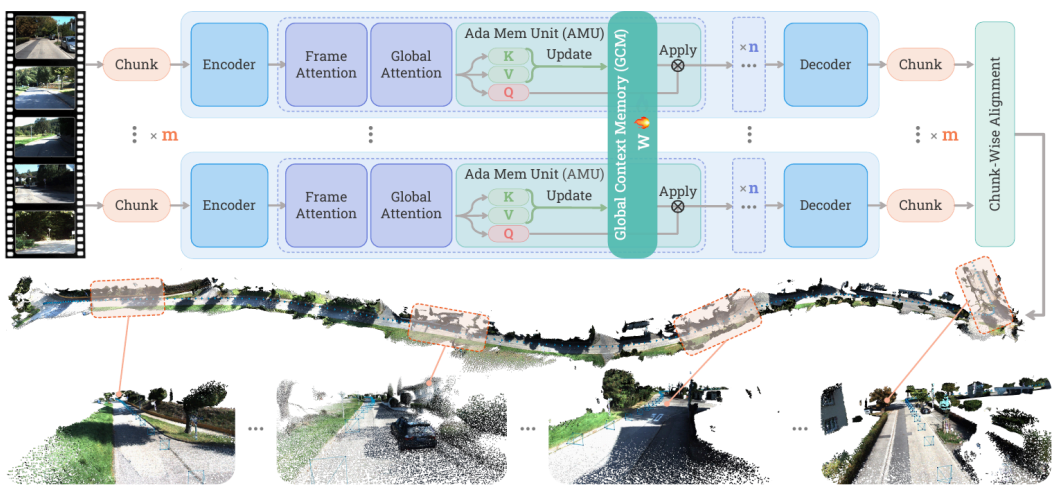



Figure 2. **Overview of Scal3R.** Our model takes a long sequence of RGB images as input and reconstructs the 3D scene within a unified

# Failure modes

Classical systems fail when:

- there is little texture,
- repeated structures confuse matching,
- moving objects dominate,
- the camera has too little baseline,
- calibration is wrong.

Learned systems add new issues:

- domain shift,
- high compute cost,
- confidence calibration,
- opaque failure cases,
- memory limits on long sequences.

> Geometry is still the diagnostic tool. Check residuals, tracks, observability, and graph structure.

# Reference materials

**LECTURE DECKS**

MIT VNAV: VO/VIO, place recognition, BoW, SLAM sparsity, factor graphs, dense reconstruction.
UCSD CSE252C: ORB-SLAM2 system overview.
SLAM Handbook: visual SLAM chapters and diagrams.

**PAPERS**

Mur-Artal and Tardos: ORB-SLAM2.
Campos et al.: ORB-SLAM3.
Engel et al.: LSD-SLAM and DSO.
Teed and Deng: DROID-SLAM.
Wang et al.: VGGT.
Maggio, Lim, Carlone: VGGT-SLAM.
Wang et al.: pi3.
Xie et al.: Scal3R.